E-Commerce Analytics with RFM Segmentation and Forecasting

Bonagani Prathusha
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad–500043
Telangana, India
Email: prathushabonagani@gmail.com

M. Haswanthi
Department of Computer Science and Engineering
KLH CSE Bowrampet Campus, KL (Deemed to be) University
Telangana, India
Email: 2420030143@klh.edu.in

Kunchala Naga Chaitanya Sphoorthy
Department of Computer Science and Engineering
KLH CSE Bowrampet Campus, KL (Deemed to be) University
Telangana, India
Email: 2420030510@klh.edu.in

Kolapally Vasavi
Department of Computer Science and Engineering
KLH CSE Bowrampet Campus, KL (Deemed to be) University
Telangana, India
Email: 2420030785@klh.edu.in

***Abstract*—**E-commerce businesses collect a lot of transaction data, yet turning those records into something a manager can act on is still hard. RetailIQ is an analytics system that takes raw retail transactions and delivers customer and sales insight. It normalizes the schema, cleans the data, builds revenue and time features, and then analyzes sales and customers. Customers are segmented by Recency, Frequency and Monetary (RFM) scores, each segment gets a retention action, and a linear regression model projects monthly revenue. The primary dataset is the UCI Online Retail dataset, with 541,909 transaction records. After cleaning, 392,692 records and 4,338 customers remain. The customers fall into six segments: Champions, Loyal, Regular, New, At Risk and Lost. A rule-based retention engine attaches a set of actions to each segment. Users start on a drag-and-drop upload page. A six-page Streamlit dashboard then shows key performance indicators, sales trends, customer segments, product and country analysis, and revenue forecasts. It also includes an assistant that answers questions in either a rule-based mode or a Gemini-based mode. The dashboard has light and dark themes, a live clock and a fixed layout that requires no scrolling. RetailIQ runs natively or in Docker.

***Index Terms*—**E-Commerce Analytics, RFM Analysis, Customer Segmentation, Revenue Forecasting, Business Intelligence, Streamlit, Retail Analytics, Generative AI

I. Introduction

Online stores generate transaction logs full of orders, products, customers, prices, quantities and purchase dates. The logs have useful information, but they do not answer the questions a business asks. Which customers are worth the most? Which ones need a retention effort? Which products earn the most? How will sales move next? Retention matters because small gains in customer loyalty can boost profits substantially [1]. Spreadsheets handle basic descriptive statistics, yet they get hard to maintain as transaction volume and the number of questions grow, and businesses need tools that turn raw transactions into customer and sales information without a dedicated enterprise analytics setup.

RetailIQ is one application that runs the whole pipeline. It covers preprocessing, feature engineering, sales analysis, RFM segmentation, retention recommendations, revenue forecasting, an interactive dashboard and a conversational assistant.

A. Problem Statement

Retail transaction files are large, noisy and unlabelled. They contain duplicate rows, cancelled orders, missing customer identifiers and invalid prices, and different sources name the same columns differently. Small and medium retailers usually lack the tooling to clean such data, find their valuable and at-risk customers, see how products and markets perform, and project revenue, all in one place.

B. Objectives

The objectives of this work are (i) to build a pipeline that cleans raw transaction data and derives revenue and time features; (ii) to segment customers into interpretable behavioural groups with RFM analysis; (iii) to link each segment to a concrete retention or win-back action; (iv) to project monthly revenue with an interpretable regression model; (v) to present the results in an interactive dashboard that starts from a drag-and-drop upload; and (vi) to add a natural-language assistant and a portable Docker deployment.

II. Methodology

Dataset. The primary dataset is the UCI Online Retail dataset [2], which holds 541,909 transaction records from a UK-based non-store online retailer between 1 December 2010 and 9 December 2011. Each record represents one product line of an invoice, and the eight attributes are listed in Table I. Most customers of this retailer are wholesalers, so a single order often includes many units of the same product, which is why revenue per order is high compared with a typical consumer shop.

TABLE I
Online Retail Dataset Attributes

Attribute	Description
InvoiceNo	Transaction identifier
StockCode	Product identifier
Description	Product description
Quantity	Number of units purchased
InvoiceDate	Transaction date and time
UnitPrice	Price per unit
CustomerID	Customer identifier
Country	Customer country

Schema normalization. Different versions of online-retail data use different column names. The Online Retail II release,

for example, uses Invoice, Price and Customer ID [3]. RetailIQ first renames incoming columns to the standard schema with an alias map, and then checks that the required columns are present. A file that lacks them is rejected immediately with a message naming the missing columns, so the user sees the problem on the upload page and it does not fail later in the pipeline.

Data preprocessing. The raw data contain duplicate records, missing customer identifiers, cancelled transactions and non-positive prices. The cleaning step does the following:

1) removes duplicate records;
2) removes transactions with a missing CustomerID, because RFM is computed per customer;
3) excludes cancelled and negative-quantity transactions;
4) removes records with a zero or negative unit price;
5) converts InvoiceDate to a datetime format; and
6) creates transaction-level revenue as Quantity $\times$ UnitPrice.

After preprocessing, 392,692 of the original 541,909 records remain, about 72.5% of the data, and the other 149,217 were removed. Feature engineering then extracts Year, Month, Day and Hour from InvoiceDate. These features feed the sales-trend charts, the day-of-week and hour-of-day charts, and the monthly aggregation used for forecasting. Table II summarizes the cleaned dataset, and the overall workflow from ingestion to the dashboard and assistant is shown in Fig. 1.

TABLE II
SUMMARY OF THE CLEANED DATASET

Measure	Value
Original records	541,909
Cleaned records	392,692
Unique customers	4,338
Unique products	3,665
Total orders	18,532
Total revenue (GBP)	8,887,209
Average order value (GBP)	480

Sales analytics. From the cleaned data the system computes total revenue, total orders, total customers and the average order value, which is total revenue divided by total orders. It also aggregates revenue by month to show the sales trend, and counts orders by day of the week and by hour of the day to show when customers buy. The percentage badges on the KPI cards compare the last month with the one before it. These figures give a first picture of the business before any customer-level modelling.

Customer segmentation. RFM analysis comes from direct-marketing practice [4]. It describes each customer by recency (how recently they bought), frequency (how often they buy) and monetary value (how much they spent), which are easy to interpret. RFM values have been clustered with K-Means and Fuzzy C-Means [5], and RFM has been combined with K-Means clustering and decision trees on online retail data [6]. Clustering needs the number of clusters to be chosen and yields groups that are harder to explain to business users, thus RetailIQ uses quintile scoring. For each customer, recency is the number of days between the customer's last purchase and a reference date just after the last transaction in the data, frequency is the number of distinct invoices, and monetary value is the total revenue. Each measure is converted to a score from 1 to 5, with recency reversed because fewer days since the last purchase is better. Rules on the recency score r, frequency score f and monetary score m then assign six segments, checked in this order: Champions ($r, f, m \geq 4$); Loyal Customers ($r, f, m \geq 3$); New Customers ($r \geq 4$ and $f \leq 2$); At Risk ($r \leq 2$ and $f \geq 3$); Lost Customers ($r \leq 2$ and $f \leq 2$); and Regular Customers for every remaining case. Table III describes what each segment looks like.

TABLE III
BEHAVIOURAL MEANING OF THE RFM SEGMENTS

Segment	Typical behaviour
Champions	Bought recently, buy often and spend the most
Loyal Customers	Repeat buyers with good spending
Regular Customers	Moderate recency, frequency and spending
New Customers	Purchased recently but only a few times
At Risk	Previously active buyers who have not bought for some time
Lost Customers	Long inactive with low engagement

On the cleaned dataset, the 4,338 customers fall into the six segments as shown in Table IV. Lost customers are the largest group, and Champions and Loyal customers together make up about 40% of customers.

TABLE IV
RFM CUSTOMER SEGMENT DISTRIBUTION

Segment	Customers	Share (%)
Lost Customers	1065	24.6
Champions	957	22.1
Loyal Customers	764	17.6
At Risk	643	14.8
Regular Customers	590	13.6
New Customers	319	7.4
Total	4338	100

Retention recommendations. A rule-based retention engine ties each segment to an action. Champions are rewarded and involved, Loyal customers get loyalty benefits, New customers get an onboarding offer, and At Risk and Lost customers get win-back or reactivation campaigns. For the selected segment, the dashboard also shows the number of customers, the revenue at stake and the average days since the last order. Behaviour sets the segment, and the segment sets the action.

Products and countries. Products are ranked by revenue and by units sold, and the top products are shown in a gallery and a table. Revenue is also aggregated by country, which shows how much the business depends on its home market. When exactly one country is selected in the sidebar, revenue is shown in that country's currency, and otherwise the base

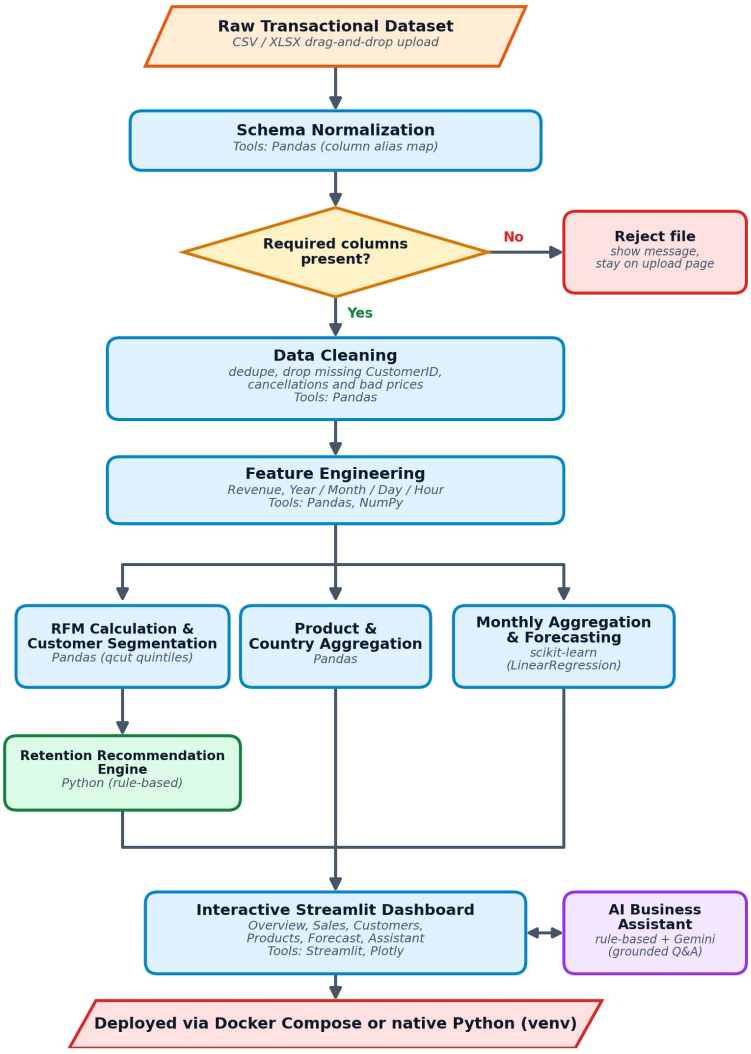


Fig. 1. End-to-end RetailIQ processing flowchart, from raw dataset ingestion to deployment.

currency (GBP) is used. Table V lists the five highest-revenue products in the dataset.

TABLE V
TOP PRODUCTS BY REVENUE

Product	Revenue (GBP)	Units
Paper Craft, Little Birdie	168,470	80,995
Regency Cakestand 3 Tier	142,265	12,374
White Hanging Heart T-Light Holder	100,392	36,706
Jumbo Bag Red Retrospot	85,041	46,078
Medium Ceramic Top Storage Jar	81,417	77,916

Revenue forecasting. Monthly revenue is aggregated from the cleaned data, and a linear trend model is trained on a month index t and the revenue y_t of that month. A linear trend is a transparent baseline [7], and Scikit-learn [8], which is widely used for predictive analysis [9], has a well-tested implementation [10]:

$$\hat{y}_t = \beta_0 + \beta_1 t. \tag{1}$$

The trained model projects revenue for 1 to 6 future months, chosen by the user in the dashboard. It is meant as an interpretable trend baseline, and the current implementation does not compute forecast-accuracy metrics such as MAE, RMSE, MAPE or R^2. Table VI gives the forecast for the next three months, which are model-based projections and not guaranteed future sales.

TABLE VI
EXAMPLE MONTHLY REVENUE FORECAST

Month	Predicted Revenue (GBP)
January 2012	926,790.85
February 2012	961,527.91
March 2012	996,264.97

AI assistant. The assistant has two modes. The rule-based mode answers fixed-intent questions such as top products, customer segments, best month and average order value, and requires no API key. The AI mode uses the Gemini API [11] for open-ended business questions. In this mode the model receives a compact summary of the computed results (KPIs, segments, top products, countries and monthly sales) and not the raw file, thus its answers stay grounded in the analysis and it cannot execute arbitrary code. Hosted language-model services are sometimes overloaded, so errors such as HTTP 503 (high demand) and rate limiting are treated as temporary. The request is retried up to three times with a short back-off, and if the main model stays busy, an alternative model is tried. An invalid API key is reported immediately and not retried. If every attempt fails, the user is told the service is busy and can switch to the rule-based mode, which does not depend on the external service.

III. System Architecture

RetailIQ has five layers: ingestion, processing, intelligence, presentation and deployment (Fig. 2). Data flows downward from one layer to the next, so each layer can change without affecting the others, and the analytical layers can be tested without the interface.

Ingestion layer. It accepts CSV and XLSX files through the Streamlit uploader [12] and normalizes column names, so schemas such as InvoiceNo versus Invoice are handled. Pandas [13] and OpenPyXL read the files, and a file that is not a valid online-retail export is rejected with an explanatory message.

Processing layer. It handles cleaning, feature engineering, RFM segmentation, product and country analytics and revenue forecasting, using Pandas, NumPy [14] and Scikit-learn. Its outputs are the cleaned transactions, the RFM table with a segment for every customer, the product and country tables, the monthly sales and the forecast.

Intelligence layer. It holds the rule-based chatbot, the retention and win-back engine and the AI business assistant, which is built on the Google GenAI SDK and a Gemini Flash model. All three work on the processing layer's results and not on the raw file.

Presentation layer. It is an upload-first, multi-page dashboard (Overview, Sales Analytics, Customers, Products & Countries, Forecast, AI Assistant) built with Streamlit and Plotly [15], with custom CSS theming for the light and dark themes.

Deployment layer. The same application runs from a native Python virtual environment or from a Docker container. Settings such as the API key come from environment variables, therefore no secret has to be written into the code.

IV. Implementation

RetailIQ is written in Python. Pandas and NumPy manage the data, Scikit-learn trains the regression model, Plotly renders the interactive charts, Streamlit serves the dashboard, and the Google GenAI SDK connects to Gemini. The code is split into modules for data cleaning, feature engineering, customer

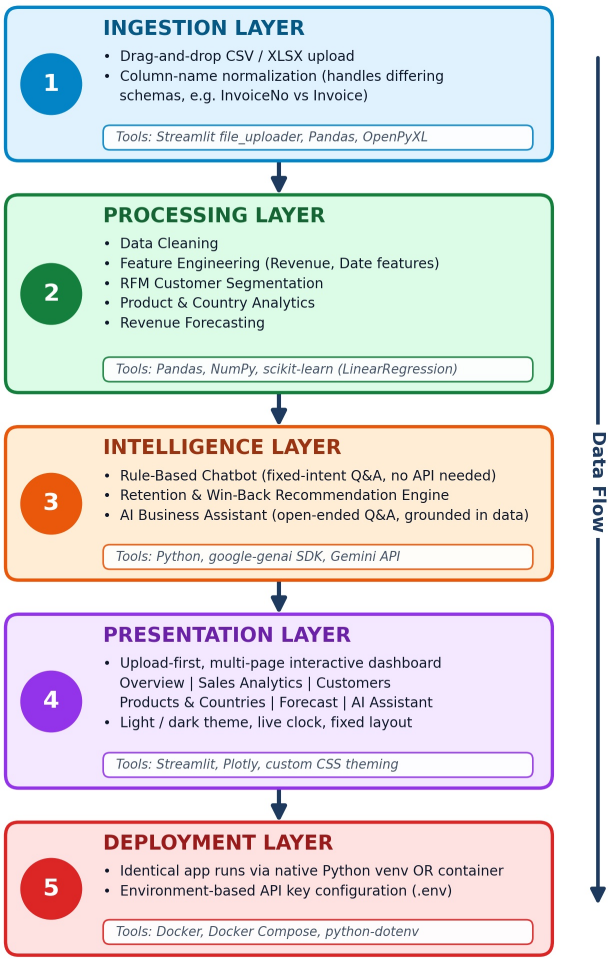


Fig. 2. RetailIQ five-layer system architecture.

analysis, segmentation, retention, forecasting, product analysis, product icons and images, currency handling and the two assistants. The Streamlit application and its theme sit in a separate dashboard folder, so the analytical modules can be tested and reused without the interface.

Uploaded files are read with the Streamlit file uploader and pass through the same normalization, validation and cleaning as the bundled sample dataset. Results are cached with Streamlit's caching, so cleaning and analysis are not repeated when the user only switches pages. The sidebar holds a date-range filter and a country filter, and every page, including the forecast and the assistant, recomputes from the filtered data. Each product in the gallery gets an icon or photo chosen from keywords in its description.

The application is deployed with Docker and Docker Compose [16]. The compose file builds the image, exposes port 8501, mounts the data folder so new datasets can be added without a rebuild, passes the optional Gemini key as an environment variable and restarts the container automatically. One command starts everything: docker compose up –build. The same code also runs natively with pip install -r require-

ments.txt and streamlit run dashboard/app.py. The API key is read from an environment variable or entered for the current session only, and a local .env file is excluded from version control.

The interface uses a soft pink and purple palette with a light and a dark theme switched by a toggle in the top bar, and a live date and time that updates every second. The layout is fixed to the browser window, and charts and tables resize to the available height, so each page fits on one screen with no scrolling. The user first sees only the upload page, and once a valid file is accepted, or the sample dataset is chosen, the full dashboard opens with six pages. A sidebar button returns to the upload page to analyze another file. Fig. 3 shows the dashboard in dark mode.

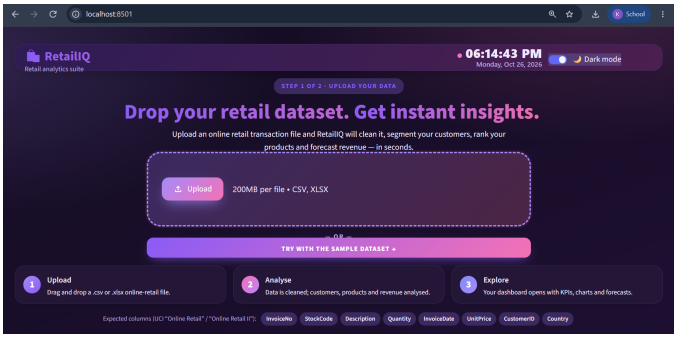


Fig. 3. Overview page in dark mode, showing the theme toggle and the live date and time.

The *Overview* page has four KPI cards (total revenue, total orders, total customers, average order value), a monthly sales trend, a monthly sales-target ring, the revenue share of the six customer segments, the top-selling products and module cards whose buttons open the other pages (Fig. 4).

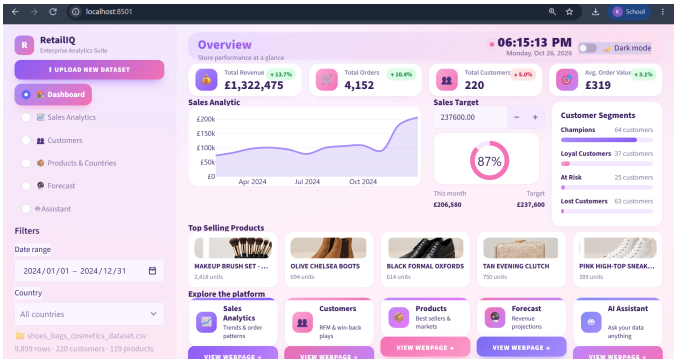


Fig. 4. RetailIQ Overview page for the UCI Online Retail dataset.

The *Sales Analytics* page shows the monthly revenue trend, orders by day of week and orders by hour of day (Fig. 5).

The *Customers* page has customer KPIs, a segment-breakdown chart, an RFM customer table and a retention tab with recommendations for each segment (Fig. 6).

The *Products & Countries* page has a gallery of the top products, a top-10 product table and a country revenue chart and table (Fig. 7).

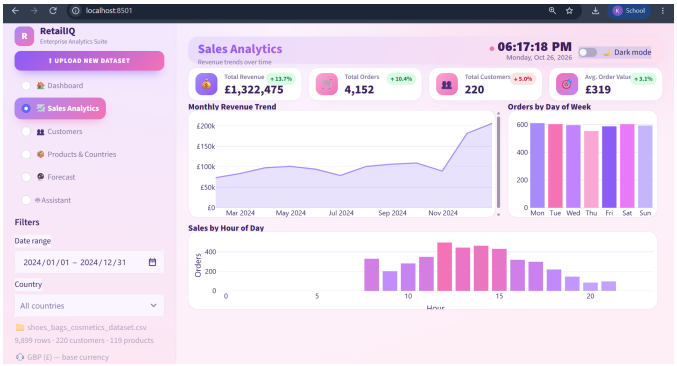


Fig. 5. Sales Analytics page: monthly revenue, orders by weekday and orders by hour.

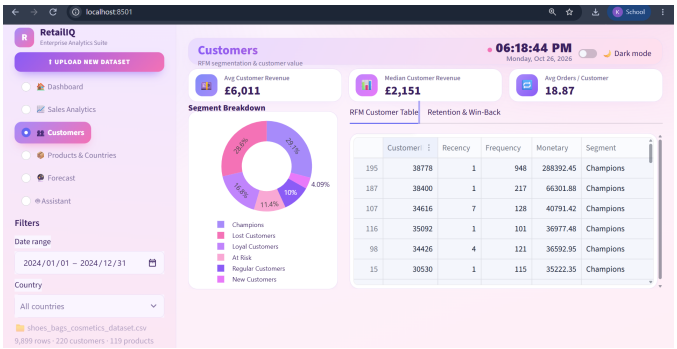


Fig. 6. Customers page: segment breakdown and RFM customer table.

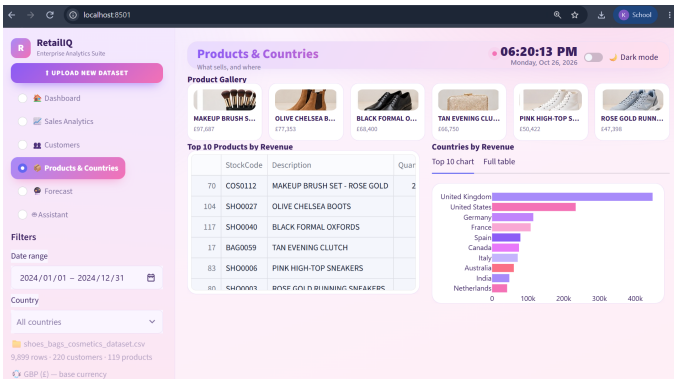


Fig. 7. Products & Countries page: product gallery, top-10 products and country revenue.

The *Forecast* page shows historical and predicted revenue, a forecast table and one card per forecast month, with a horizon of 1 to 6 months (Fig. 8).

The *AI Assistant* page is a chat interface with suggested questions and a switch between rule-based and AI mode (Fig. 9).

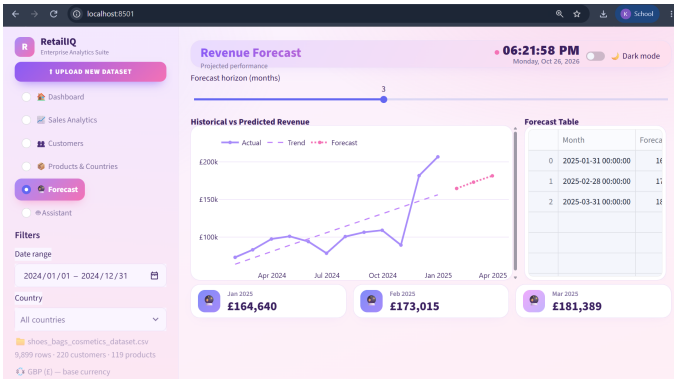


Fig. 8. Forecast page: actual revenue, fitted trend and projected revenue.

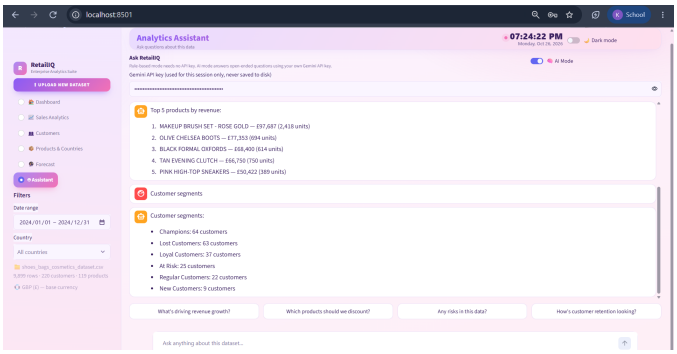


Fig. 9. AI Assistant page with suggested questions and sample answers.

V. Results and Conclusions

The main page of RetailIQ is the upload page (Fig. 10): the user drops a CSV or XLSX file, or tries the sample dataset, and the pipeline then cleans the data and opens the dashboard. On the UCI Online Retail dataset, 541,909 records give 392,692 cleaned records (Table II) covering 4,338 customers, with total revenue of about 8.89 million GBP from 18,532 orders and an average order value of about 480 GBP.

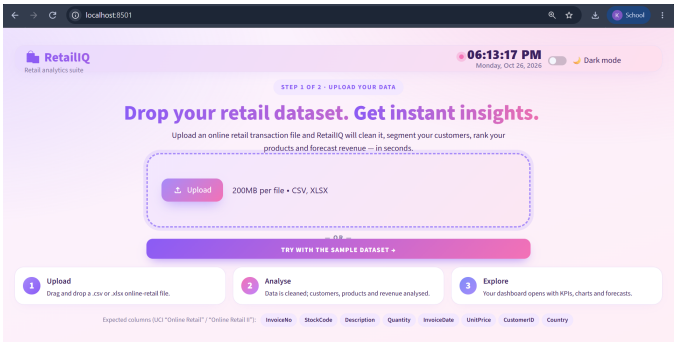


Fig. 10. Upload page of RetailIQ: drag-and-drop area, sample-dataset option and a three-step guide.

The customers fall into six RFM segments (Table IV). The customers fall into six RFM segments (Table IV), and At Risk customers are the main target of win-back actions. The United Kingdom contributes most of the revenue, and the linear model projects a steady upward trend for January to March 2012 (Table VI).

Automated smoke tests opened all six pages in both themes without runtime errors, invalid files were rejected with a clear message, and the assistant's retry logic was tested with simulated service errors. To conclude, RetailIQ turns raw transaction data into customer, product, sales and revenue insight in one lightweight Python application. It pairs each RFM segment with an action, adds a revenue forecast and lets non-technical users query the results in natural language. Because the design starts from an upload, the same pipeline can be reused on any compatible retail file, and Docker makes the system portable.

VI. Future Work

RetailIQ works well as a first version, yet it has some limits. The points below explain each limit in simple words and what can be done next.

- **Better forecasting.** The forecast only follows a straight trend and ignores seasonal ups and downs. Stronger models such as ARIMA, Prophet, XGBoost or LSTM can be tried.
- **Check forecast accuracy.** The system does not yet measure how wrong the forecast can be. Standard error measures (MAE, RMSE, MAPE and R^2) can be added.
- **Incomplete last month.** The data stops on 9 December 2011, thus the last month looks lower than it really is. The system can mark or leave out an incomplete month.
- **Clean the product list.** Items such as postage and manual adjustments show up as products. They can be filtered out so only real products are ranked.
- **More types of data.** One country (the United Kingdom) dominates the data, and files without the required columns are rejected. Support for more dataset formats can be added.
- **Smarter offline assistant.** AI mode needs a Gemini API key and internet, and the rule-based mode answers only fixed questions. The rule-based mode can be extended with more question types.
- **Predict customers who may leave.** Machine learning can predict churn and the lifetime value of each customer.
- **Find unusual sales.** Unusual orders and sudden changes in sales can be detected automatically.
- **Explain predictions.** Explainable-AI methods can show why the model made a prediction.
- **Cloud and recommendations.** The system can be hosted in the cloud for many users, and better product and customer recommendations can be added.

References

[1] F. F. Reichheld and W. E. Sasser, Jr., "Zero defections: Quality comes to services," *Harvard Business Review*, vol. 68, no. 5, pp. 105–111, Sep.–Oct. 1990.
[2] D. Chen, "Online Retail," UCI Machine Learning Repository, 2015, doi: 10.24432/C5BW33.
[3] D. Chen, "Online Retail II," UCI Machine Learning Repository, 2012, doi: 10.24432/C5CG6D.
[4] A. M. Hughes, *Strategic Database Marketing*. Probus Publishing, 1994.

[5] A. J. Christy, A. Umamakeswari, L. Priyatharsini and A. Neyaa, "RFM ranking – An effective approach to customer segmentation," *J. King Saud Univ. – Comput. Inf. Sci.*, vol. 33, no. 10, pp. 1251–1257, Dec. 2021.

[6] D. Chen, S. L. Sain and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," *J. Database Marketing & Customer Strategy Management*, vol. 19, no. 3, pp. 197–208, 2012, doi: 10.1057/dbm.2012.17.

[7] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[9] K. S. Madhuri, B. Prathusha, G. M. Reddy, G. Deshmukh, K. B. S. Kumar and A. Markos, "Performance analysis of machine learning algorithms by using WEKA and Scikit-learn," in *Data Science and Big Data Analytics (IDBA 2023)*, D. Mishra, X.-S. Yang, A. Unal and D. S. Jat, Eds. Singapore: Springer, 2024, pp. 633–644, doi: 10.1007/978-981-99-9179-2_47.

[10] Scikit-learn Developers, "LinearRegression documentation," Scikit-learn Documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

[11] Google, "Gemini API documentation," Google AI for Developers. [Online]. Available: https://ai.google.dev/gemini-api/docs

[12] Streamlit, "Streamlit documentation." [Online]. Available: https://docs.streamlit.io

[13] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61, doi: 10.25080/Majora-92bf1922-00a.

[14] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[15] Plotly Technologies Inc., "Collaborative data science," Montreal, QC, 2015. [Online]. Available: https://plot.ly

[16] Docker, "Docker documentation." [Online]. Available: https://docs.docker.com